

3. Bubble Sort with Big-O Analysis

Aim

To implement **Bubble Sort using Python** and analyze its time and space complexity using Big-O notation.

Procedure

1. Create an array of unsorted elements.
2. Compare two adjacent elements.
3. If the first element is greater than the second element, swap them.
4. Continue comparing adjacent elements throughout the array.
5. Repeat the process for multiple passes.
6. After each pass, the largest unsorted element moves to its correct position.
7. Continue until the entire array is sorted.
8. Display the sorted array.
9. Analyze the time and space complexity.

main.py	Visualize	Run ▶	Output
<pre>1 def bubbleSort(arr): 2 n = len(arr) 3 for i in range(n): 4 for j in range(0, n - i - 1): 5 if arr[j] > arr[j + 1]: 6 arr[j], arr[j + 1] = arr[j + 1], arr[j] 7 return arr 8 arr = [64, 34, 25, 12, 22, 11, 90] 9 print("Before Sorting:", arr) 10 bubbleSort(arr) 11 print("After Sorting:", arr)</pre>			Before Sorting: [64, 34, 25, 12, 22, 11, 90] After Sorting: [11, 12, 22, 25, 34, 64, 90]

Result

Thus, the Python program successfully sorts the given array using the **Bubble Sort algorithm**.